

Why training is expensive, and where the money goes

Four budgets, one formula, and a whiteboard answer you can defend to finance

The meeting was about something else entirely when the director turns to you: "*Ballpark — what would it cost us to train our own 7B from scratch?*" Everyone waits. The honest answer fits on a whiteboard in four lines, and by the end of this chapter you will write them without notes: **$6 \times 7B \times 2T \text{ tokens} \approx 8.4 \times 10^{22} \text{ FLOPs}$; at 40% utilization of an H100 that's ~59,000 GPU-hours; at \$2.50 an hour, ~\$147K for the run; multiply by ~1.4 for everything that isn't the run — call it \$205K, eleven days on 256 GPUs.** Every number in that answer comes from one formula and three exchange rates, and every efficiency technique in this book is an attack on one of its factors. This chapter is the accounting: what the budgets are, how they convert into each other, where the published frontier runs landed, and — the part the whiteboard hides — where the money actually leaks, which is almost never where new hires first look.

6.1 The four budgets, and the exchange rates between them

Training spends four currencies at once, and confusion between them is the root of most bad planning conversations. **FLOPs** are the physics: floating-point operations the mathematics demands, fixed by model and data before any hardware is chosen. **Tokens** are the diet: how much (deduplicated, filtered — Chapter 10) text exists to learn from, often the binding constraint before money is. **Wall-clock** is the calendar: the same FLOPs spread over more GPUs finish sooner, and calendar time is opportunity cost — a model three months late into this market lost value no efficiency recovers. **Dollars** are what finance sees, and the only currency the other three must eventually be quoted in. The exchange rates are the whole game:



The three exchange rates, and who owns each:

6N FLOPs per token — set by the model you chose (§6.3); the architect's number

peak x MFU — what your hardware and software actually deliver (Ch. 8); the engineer's number, 30–45% in good runs

\$/GPU-hour — procurement's number, and it varies ~3x with commitment terms; efficiency work and price negotiation compou

Every chapter of Parts III–V attacks one arrow: better data shrinks D, better architecture shrinks 6N, better systems i

Figure 6.1 — The currency chain. Tokens become FLOPs through the model's size, FLOPs become GPU-hours through achieved utilization, GPU-hours become dollars through the rental rate — and become calendar through cluster size. Fluency in these conversions is what makes you the person who answers the director's question.

6.2 A real run, line by line

Now the whiteboard answer, itemized the way an invoice would be. The scenario model: 7B parameters, 2T tokens — a sensible overtrained small model in Chapter 5's sizing logic — on rented H100s at \$2.50/hour, 40% MFU (a good, honest number; Chapter 8 is about earning it):

Table 6.1 — The 7B @ 2T run, fully loaded. The headline line is barely 70% of the real bill — the multiplier below it is where estimates die.

LINE ITEM	AMOUNT	HOW IT'S COMPUTED
The run itself: 8.4×10^{22} FLOPs	\$147,500	$6.7e9 \cdot 2e12 \div (989 \text{ TF} \times 0.40) = 59,000 \text{ H100-h} \times \2.50
Ablations & experiments that shaped it	\$36,900	~25% of the run — data mixes, LR sweeps at small scale (Ch. 7), the runs that didn't ship
Failures & restarts	\$10,300	~7% — the every-few-hours reality of §4.3, minus what checkpoints save
Evaluation during training	\$4,400	~3% — Chapter 9's cadence, priced in
Post-training (SFT + preferences)	\$2,900	~2% — Chapter 5's inversion, on the invoice
Storage & data pipeline	\$3,000	checkpoints (~100 GB each), 8 TB corpus, CPU preprocessing, months of retention

LINE ITEM	AMOUNT	HOW IT'S COMPUTED
Total	≈ 1.39× the naive number · ~11 days of calendar on 256 GPUs \$205,000	

Three readings of the table before we derive its first line. The **multiplier is structural, not sloppiness**: ablations are how the run got good (skipping them buys a worse model, not a cheaper one), and failure overhead is physics at cluster scale — budget 1.3–1.5× or be surprised. The **people are missing**: months of engineer time typically rivals the compute line at this scale, and it is the line that efficiency skills — the subject of this book — actually shrink. And the **same table at other scales** is just proportional arithmetic on line one, which is why §6.3's formula matters more than any single row: master it and every budget question becomes multiplication.

6.3 FLOPs per token: the 6ND rule, derived honestly

The formula deserves five minutes of respect, because you will use it weekly for years. Where does $C \approx 6ND$ come from? Start with the forward pass. Almost all of a transformer's work is matrix multiplies (Chapter 3), and a matmul that touches a weight uses it in exactly one multiply and one add per token: **2 FLOPs per parameter per token**. Forward cost: $2N$ per token. The backward pass computes two gradients per weight-use — one with respect to the activations (to keep propagating) and one with respect to the weight itself — each again a multiply-add: **4 FLOPs per parameter**. Total: $6N$ per token, $6ND$ for the run.¹ The same accounting gives inference its rule of thumb, $2N$ per generated token — one forward, no backward — which is where Chapter 7's training-versus-serving trade will get its numbers.

What the rule ignores, and when that bites: **attention's pairwise term**. The $qk^\top$ and score-times-value products don't touch weights, so 6ND misses them; the correction is roughly a factor $(1 + \frac{L}{6d})$ — negligible at short context, +33% for our 7B at 8K ($L/6d = 8192/24576$), and dominant past Chapter 13's crossover $L^* \approx 6d$. Budget with 6ND below ~8K; add the surcharge above it; switch to per-stage accounting for Chapter 21's ladders. In code, the whole section is:

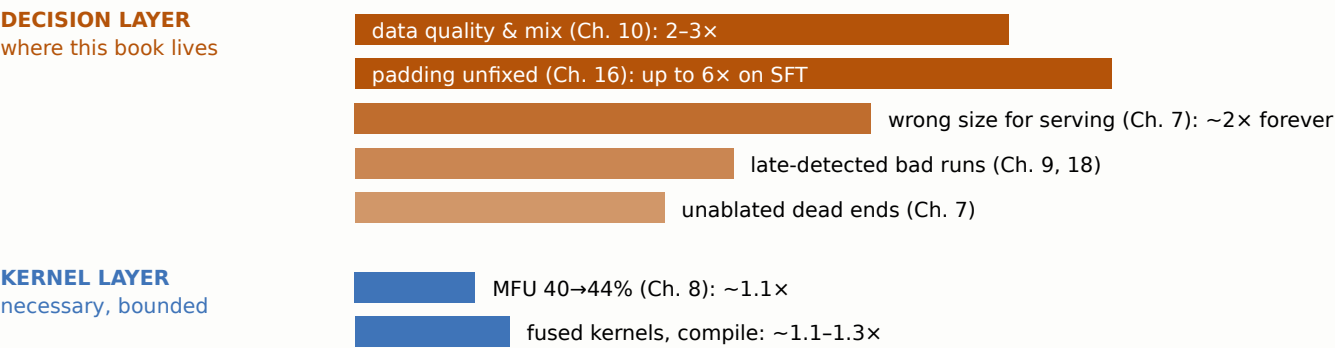
```
# the budget calculator – keep it in your shell profile
def training_budget(N, D, L=4096, d_model=4096, peak_tflops=989,
                    mfu=0.40, dollars_per_hour=2.50, overhead=1.4):
    flops = 6 * N * D * (1 + L / (6 * d_model))    # 6ND + attention surcharge
    gpu_hours = flops / (peak_tflops * 1e12 * mfu) / 3600
    return {"flops": f"{flops:.2e}",
            "gpu_hours": round(gpu_hours),
            "run_cost": round(gpu_hours * dollars_per_hour),
            "loaded_cost": round(gpu_hours * dollars_per_hour * overhead),
            "days_on_256": round(gpu_hours / 256 / 24 * 1.15, 1)}

training_budget(7e9, 2e12)    # -> the whiteboard answer of $6.2
training_budget(70e9, 15e12) # -> why the next size up is a different meeting
# inference: ~2N FLOPs per generated token (one forward, no backward)
```

6.4 Where the waste is: decisions, not kernels

Now the premise this book announced in its title, cashed out with the currency chain in hand. When engineers first meet the budget, instinct sends them to the kernels: squeeze MFU from 40% to 44%, save 10%. That work is real and Chapter 8 honors it. But look at the sizes of the leaks the decision layer controls. A mediocre data mix against a good one is worth more loss than a year of kernel work — the difference between training on raw crawl and on Chapter 10's filtered corpus has been measured repeatedly at "several times the tokens for the same quality," a 2–3× lever on D . Padding, before Chapter 16, was silently multiplying token cost by up to 6× on instruction data. A wrong model size for the serving reality (Chapter 7) taxes every token the model ever produces, forever. An unnoticed bad run — divergence discovered a week late because evals lagged (Chapters 9, 18) — burns pure money at cluster rate. And an abandoned direction that better small-scale ablations would have killed early (Chapter 7's scaling-law fits) costs a full experiment line-item. Stack them and the picture is stark:

TYPICAL SIZE OF EACH LEVER (multiplier on cost-for-equal-quality; bars illustrative, cites in text)



Both layers compound — the argument is priority, not either/or. But no kernel recovers the cost of training the wrong thing well.

Figure 6.2 — The waste taxonomy. Kernel-layer wins are real, bounded, and someone else's library often delivers them; decision-layer wins are larger, compound, and are yours alone to take or miss. This asymmetry is the book's thesis stated as a bar chart.

6.5 What the big labs do: the published ledgers

Table 6.2 — Compute budgets the labs told us about. Sorted by FLOPs; dollar figures only where the lab itself published one.

RUN	FLOPS	GPU-HOURS	WHAT THE DISCLOSURE TEACHES
GPT-3 175B (2020) ²	3.1×10 ²³	—	The number that started budget culture; ~\$4–5M of 2020 cloud compute for what a 2025-recipe 7B outperforms — recipes beat raw scale over time
Llama 2 7B (2023) ³	~7.6×10 ²²	184,320 A100-h	Almost exactly our \$6.2 scenario on older silicon — sanity-checks the whiteboard math

RUN	FLOPS	GPU-HOURS	WHAT THE DISCLOSURE TEACHES
DeepSeek-V3 671B MoE (2024) ⁴	$\sim 3.4 \times 10^{24}$	2.788M H800- h · \$5.576M	The efficiency shock: frontier scale at a published $\sim \$5.6\text{M}$ via MoE sparsity, FP8, and MLA — Parts III–IV applied in anger
Llama 3.1 405B (2024) ⁵	3.8×10^{25}	30.84M H100- h 1.4× —	Dense frontier honesty: at $\sim \$2/\text{h}$ that's $\sim \$60\text{M}+$ before the 419 interruptions ($\$4.3$) are in there

Two lessons hide in the column gaps. Between GPT-3 and Llama 2 7B: *five years of recipe improvements bought $\sim 100\times$ cost reduction for equal capability* — data quality, Chinchilla-aware sizing, architecture — almost none of it kernels. Between Llama 3.1 and DeepSeek-V3: a $\sim 10\times$ FLOPs gap closed to near-parity on many evals by aggressive architectural efficiency — the strongest public evidence that the decision layer of $\$6.4$ is where budgets are won. When someone tells you frontier training costs are a law of nature, this table is the rebuttal.

6.6 Practical considerations

Estimating your own run: the four-line discipline. Write N and D first, and justify D from data you actually possess after Chapter 10's cleaning — the commonest estimate-killer is assuming tokens you turn out not to have. Convert with $6ND$ (plus the $\$6.3$ surcharge if long-context). Use an MFU you have *measured on your stack at small scale*, not aspiration: 30% if unproven, 40%+ only with evidence. Multiply by 1.4 loaded. Then check the calendar against your actual cluster and add 15% operational drag — if wall-clock exceeds your decision horizon, the plan is wrong regardless of dollars.

Price the hours like an adult. The same H100-hour costs $\$2$ or $\$6$ depending on commitment length, spot exposure, and provider; owned hardware runs cheaper per hour but adds idle-time risk (an owned cluster you can't keep busy is the most expensive option of all). For bursty experiment phases, spot with aggressive checkpointing ($\$4.3$ makes preemption survivable) routinely halves the ablation line of Table 6.1.

Track cost per *result*, not per hour. A dashboard line of dollars-per-eval-point-gained sounds silly until it catches a month where the run spent $\$40\text{K}$ standing still — Chapter 9's evals, Chapter 18's diagnostics, and this chapter's accounting exist to make that month impossible.

Common pitfalls

Budgeting the final run only.

The 1.4× is structural ($\$6.2$). Proposals that omit it get approved, then either overrun or skip the ablations that make models good.

Peak-FLOPs arithmetic.

Dividing by 989 TF instead of $989 \times \text{MFU}$ flatters the estimate by 2.5×. Every number without a measured MFU beside it is fiction.

Confusing dataset size with D.

Epochs multiply, dedup shrinks, packing changes the token count that actually meets the model (Ch. 16). D is tokens *trained on*, post-pipeline.

Ignoring the attention surcharge at long context.

6ND under-bills a 128K run badly (§6.3); Chapter 21's staged recipes need per-stage budgets.

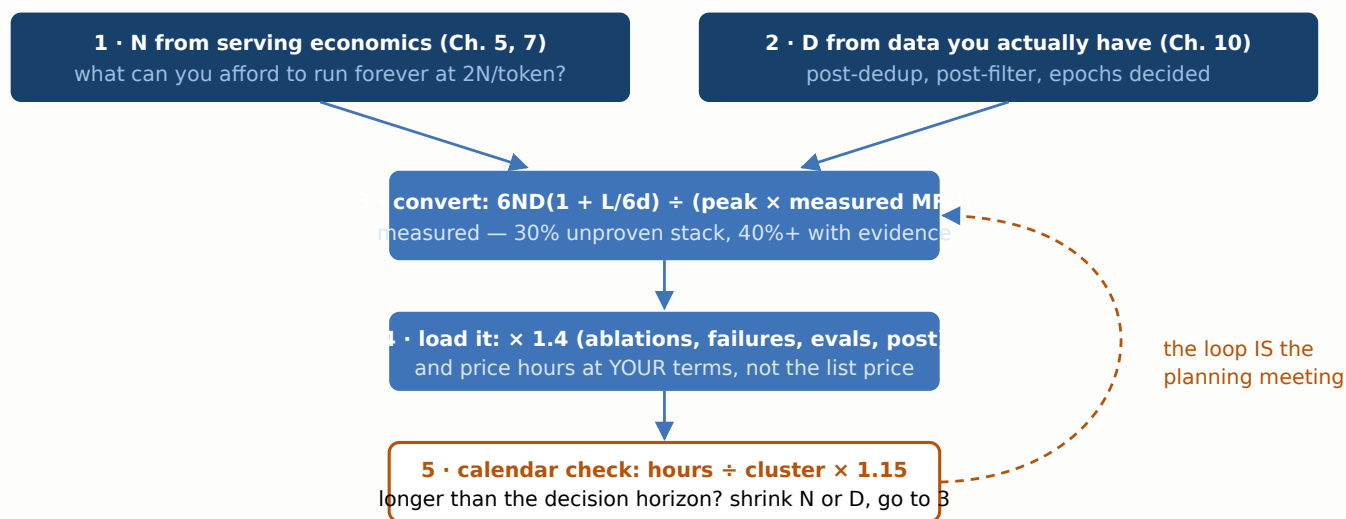
Forgetting inference in the sizing meeting.

Training is bought once; serving is bought forever at $\sim 2N$ per token. Chapter 7 makes this the sizing criterion — bring it to the meeting where N is chosen.

People off the books.

At sub-frontier scale, engineer-months rival compute. Efficiency skills are the only line item that gets cheaper with practice — the actual thesis of the program this book teaches.

6.7 Decision framework: budgeting a run



Go/no-go belongs to the loaded number and the calendar together — a run you can afford but can't wait for is still a no.

Figure 6.3 — Budgeting as a loop, not a lookup. Serving picks N, data picks D, measurement picks MFU, structure picks the multiplier — and the calendar check sends you around again until all four budgets close simultaneously.

Cost-benefit, bluntly. Learning this arithmetic is free and pays at every scale — it is the difference between negotiating with finance and being negotiated. The 1.4× honesty costs approval friction once and prevents the worse conversation later. Measuring MFU before quoting it costs a day of benchmarking and removes the largest error bar in the estimate.

How this scales: the formula is linear in N and D but the *meeting* is not — past $\sim 10^{24}$ FLOPs, wall-clock and cluster acquisition dominate dollars, failure overhead grows with fleet size, and the ablation line grows because mistakes at scale are unaffordable — which is exactly why Chapter 7 teaches you to buy your answers at small scale first.

6.8 Summary

Training spends four currencies — FLOPs, tokens, wall-clock, dollars — chained by three exchange rates: $6N$ FLOPs per token from the architecture, $\text{peak} \times \text{MFU}$ from the systems stack, and $\$/\text{GPU-hour}$ from procurement. The whiteboard answer for any run is four lines: $C = 6ND$ (plus $L/6d$ at long context), divide by honest utilization, price the hours, multiply by ~ 1.4 for the ablations, failures, evals, and post-training that are structurally part of shipping a good model — \$205K and eleven days on 256 GPUs for a 7B on 2T tokens. The published ledgers bracket the field, and their gaps teach more than their rows: five years of recipe improvements bought $\sim 100\times$ cost-per-capability over GPT-3, and DeepSeek's \$5.6M frontier run showed the decision layer beating a $10\times$ FLOPs disadvantage. That is the taxonomy to internalize: kernel-layer waste is real but bounded ($\sim 1.1\text{--}1.3\times$), while data quality, padding, sizing, and late-detected failures are multiples — decisions, not kernels, is where the money goes and where this book spends its pages. Budget as a loop until all four currencies close, quote nothing without a measured MFU beside it, and remember the line item that improves with practice: the person doing the estimating.

References

1. Kaplan, J., et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361. The $C \approx 6ND$ accounting and its derivation conventions.
2. Brown, T., et al. (2020). Language Models are Few-Shot Learners. *NeurIPS*. arXiv:2005.14165. GPT-3's 3.14×10^{23} FLOPs.
3. Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288. Per-size GPU-hour table; 184,320 A100-hours for the 7B.
4. DeepSeek-AI (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437. 2.788M H800-hours, \$5.576M at the paper's $\$/\text{GPU-hour}$ assumption.
5. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. 30.84M H100-hours and 3.8×10^{25} FLOPs for 405B pre-training.

Next: Chapter 7 — *Scaling laws for training and inference compute* — how to choose N and D so the budget above buys the most model. · [Training Efficient LLMs](#)